

Shared-memory programming using OpenMP

Oscar Vargas Pabon

All codes where uploaded to the following repository under the 2_submit folder

https://github.com/osvarp/paralelComputing_week3

1 OpenMP Model

When I run it without the '-fopenmp' flag it only prints 'hello' once. When I run it with the flag it prints 'Hello' 8 times.

```
japeto@71c48eda2307:~/app$ g++ hello.cpp
japeto@71c48eda2307:~/app$ ./a.out
Hello
World
japeto@71c48eda2307:~/app$ g++ -fopenmp hello.cpp
japeto@71c48eda2307:~/app$ ./a.out
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
World
japeto@71c48eda2307:~/app$
```

2 The team

It's possible to create odd number of threads and more threads than cores (apparently my machine has 8 threads and I tried creating >10).

```
japeto@65aec48f73ef:~/app$ g++ -fopenmp hello.cpp
japeto@65aec48f73ef:~/app$ ./a.out 2
Hello
Hello
World
japeto@65aec48f73ef:~/app$ ./a.out 4
Hello
Hello
Hello
Hello
World
japeto@65aec48f73ef:~/app$ ./a.out 8
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
World
japeto@65aec48f73ef:~/app$ ./a.out 5
Hello
Hello
Hello
Hello
Hello
World
```

3 Thread identifier

Done

```

japeto@d006bb2dd4d7:~/app$ g++ -fopenmp hello.cpp
japeto@d006bb2dd4d7:~/app$ ./a.out 2
Hello I'm thread <0>. Nice to meet you!!!!
Hello I'm thread <1>. Nice to meet you!!!!
World
japeto@d006bb2dd4d7:~/app$ ./a.out 4
Hello I'm thread <0>. Nice to meet you!!!!
Hello I'm thread <2>. Nice to meet you!!!!
Hello I'm thread <1>. Nice to meet you!!!!
Hello I'm thread <3>. Nice to meet you!!!!
World
japeto@d006bb2dd4d7:~/app$ ./a.out 8
Hello I'm thread <2>. Nice to meet you!!!!
Hello I'm thread <0>. Nice to meet you!!!!
Hello I'm thread <4>. Nice to meet you!!!!
Hello I'm thread <7>. Nice to meet you!!!!
Hello I'm thread <6>. Nice to meet you!!!!
Hello I'm thread <3>. Nice to meet you!!!!
Hello I'm thread <1>. Nice to meet you!!!!
Hello I'm thread <5>. Nice to meet you!!!!
World
japeto@d006bb2dd4d7:~/app$ ./a.out 5
Hello I'm thread <0>. Nice to meet you!!!!
Hello I'm thread <4>. Nice to meet you!!!!
Hello I'm thread <1>. Nice to meet you!!!!
Hello I'm thread <2>. Nice to meet you!!!!
Hello I'm thread <3>. Nice to meet you!!!!
World

```

4 Identifier + pragma omp parallelize for

If it's not evenly divisible, then some threads may end with more or less iterations to execute.

```
japeto@c4e068c70b24:~/app$ g++ -fopenmp hello.cpp
japeto@c4e068c70b24:~/app$ ./a.out 2
Hello I'm thread <0>, running my routine iteration <0>
Hello I'm thread <0>, running my routine iteration <1>
Hello I'm thread <0>, running my routine iteration <2>
Hello I'm thread <0>, running my routine iteration <3>
Hello I'm thread <0>, running my routine iteration <4>
Hello I'm thread <0>, running my routine iteration <5>
Hello I'm thread <0>, running my routine iteration <6>
Hello I'm thread <0>, running my routine iteration <7>
Hello I'm thread <1>, running my routine iteration <8>
Hello I'm thread <1>, running my routine iteration <9>
Hello I'm thread <1>, running my routine iteration <10>
Hello I'm thread <1>, running my routine iteration <11>
Hello I'm thread <1>, running my routine iteration <12>
Hello I'm thread <1>, running my routine iteration <13>
Hello I'm thread <1>, running my routine iteration <14>
Hello I'm thread <1>, running my routine iteration <15>
World ending by pollution of ants
japeto@c4e068c70b24:~/app$ ./a.out 5
Hello I'm thread <0>, running my routine iteration <0>
Hello I'm thread <0>, running my routine iteration <1>
Hello I'm thread <0>, running my routine iteration <2>
Hello I'm thread <0>, running my routine iteration <3>
Hello I'm thread <4>, running my routine iteration <13>
Hello I'm thread <4>, running my routine iteration <14>
Hello I'm thread <4>, running my routine iteration <15>
Hello I'm thread <2>, running my routine iteration <7>
Hello I'm thread <2>, running my routine iteration <8>
Hello I'm thread <2>, running my routine iteration <9>
Hello I'm thread <1>, running my routine iteration <4>
Hello I'm thread <1>, running my routine iteration <5>
Hello I'm thread <1>, running my routine iteration <6>
Hello I'm thread <3>, running my routine iteration <10>
Hello I'm thread <3>, running my routine iteration <11>
Hello I'm thread <3>, running my routine iteration <12>
World ending by pollution of ants
```

5 Degree of parallelization

The reason behind the errors are race conditions involving the variable where I add in each iteration.

As the COUNT amount increases, the mistakes it makes seems to increase too. For some reason, having 4 and 8 threads tend to generate a bigger error than 2, 32 or 64 in my experiments.

```
^[[A]japeto@435259ca1bf5:~/app$ g++ -fopenmp geom_series_adder.cpp
japeto@435259ca1bf5:~/app$ ./a.out
Expectation with 1000 iterations per element
+-----+-----+-----+-----+-----+
| COUNT|Thrd=  2|Thrd=  4|Thrd=  8|Thrd= 32|Thrd= 64|
+-----+-----+-----+-----+-----+
|    10| 0.001000| 0.927000| 1.000000| 0.001000| 0.004000|
+-----+-----+-----+-----+-----+
|   100| 0.018000| 0.999000| 0.997000| 0.011000| 0.005000|
+-----+-----+-----+-----+-----+
|  1000| 0.983000| 1.000000| 1.000000| 0.057000| 0.023000|
+-----+-----+-----+-----+-----+
japeto@435259ca1bf5:~/app$
```

6 Private Vs global scope

Changing it to private makes it always give wrong answers. The reason is that by making it private, then each thread calculates with it's own version of 'sum', meaning that the result among all threads is never calculated.

```
Expectation with 1000 iterations per element
```

COUNT	Thrd= 2	Thrd= 4	Thrd= 8	Thrd= 32	Thrd= 64
10	1.000000	1.000000	1.000000	1.000000	1.000000
100	1.000000	1.000000	1.000000	1.000000	1.000000
1000	1.000000	1.000000	1.000000	1.000000	1.000000

7 Reduction

Reduction tells omp that the variable 'sum' of each thread will be subject to a reduction (all will be added into a single one).

```
japeto@60c6423d6d25:~/app$ g++ -fopenmp geom_series_adder.cpp
japeto@60c6423d6d25:~/app$ ./a.out
Expectation with 1000 iterations per element
```

COUNT	Thrd= 2	Thrd= 4	Thrd= 8	Thrd= 32	Thrd= 64
10	0.000000	0.000000	0.000000	0.000000	0.000000
100	0.000000	0.000000	0.000000	0.000000	0.000000
1000	0.000000	0.000000	0.000000	0.000000	0.000000

8 Compute PI

```
.japeto@cd0a93a19ee6:~/app$ g++ cpi.cpp
japeto@cd0a93a19ee6:~/app$ time ./a.out
The value of PI is 3.141592653590

real    0m3.491s
user    0m3.485s
sys      0m0.000s
```

9 Parallelize PI

The program is not only noticeable slower, it also computes an incorrect answer to PI. My hypothesis to the second downfall is that the stability of the previous algorithm depends on the order of the summands (addition is not commutative over floating point-_-).

```
japeto@d984e494f60d:~/app$ g++ -fopenmp cpi.cpp
japeto@d984e494f60d:~/app$ time ./a.out
The value of PI is 402.123859659516

real    1m25.650s
user    11m21.969s
sys      0m0.300s
japeto@d984e494f60d:~/app$ g++ cpi.cpp
/tmp/ccKtd53P.o: In function `main':
cpi.cpp:(.text+0x2b): undefined reference to `omp_set_num_threads'
collect2: error: ld returned 1 exit status
japeto@d984e494f60d:~/app$ g++ cpi.cpp
japeto@d984e494f60d:~/app$ ./a.out
The value of PI is 3.141592653590
japeto@d984e494f60d:~/app$ time ./a.out
The value of PI is 3.141592653590

real    0m3.570s
user    0m3.564s
sys      0m0.001s
```